

SLIDES 2 - INFORMATION RETRIEVAL

#IR

L'**Information Retrieval (IR)** è il campo dell'informatica che studia come trovare informazioni rilevanti all'interno di grandi documenti non strutturati (soprattutto testo).

Alcuni esempi di applicazioni di IR includono:

- Motori di ricerca (Google, Bing, etc...);
- Ricerca nelle e-mail;
- Ricerca nei file del computer;
- Archivi aziendali;
- Database giuridici.

L'obiettivo principale è **recuperare i documenti più rilevanti rispetto al bisogno informativo dell'utente**.

L'importanza dell'IR è cresciuta esponenzialmente con l'aumento della quantità di informazioni disponibili online: attualmente circa 1.3 miliardi di siti web di cui solo 15% attivi e oltre un milione di nuovi url creati al giorno. Ciò rende fondamentale avere sistemi efficienti per filtrare e trovare le informazioni desiderate.

Importante è conoscere la differenza tra **dati strutturati** e **dati non strutturati**:

- **Dati strutturati**: organizzati in un formato predefinito (es. database relazionali, fogli di calcolo, ecc...).
- **Dati non strutturati**: non organizzati in un formato predefinito (es. pagine web, email, documenti di testo, ecc...).

Un sistema di IR considera:

- **Collection**: insieme di documenti da cui recuperare informazioni.
- **Query**: richiesta dell'utente che esprime il bisogno informativo.
- **Goal**: trovare i documenti più rilevanti rispetto alla query.

Uno dei modelli base di IR è **Bag of Words**. L'idea è che ogni documento rappresenta null'altro che un insieme di parole, senza considerare l'ordine o la grammatica. In questo senso alcune delle tecniche usate sono **stop words** (parole inutili e non significanti vengono ignorate) e **stemming** (riduzione delle parole alla loro radice).

Per capire quanto un documento è rilevante rispetto a una query, si usano diverse statistiche come la frequenza della parola in un documento (**TF - Term Frequency**) e la frequenza inversa

della parola in tutti i documenti (**IDF - Inverse Document Frequency**).

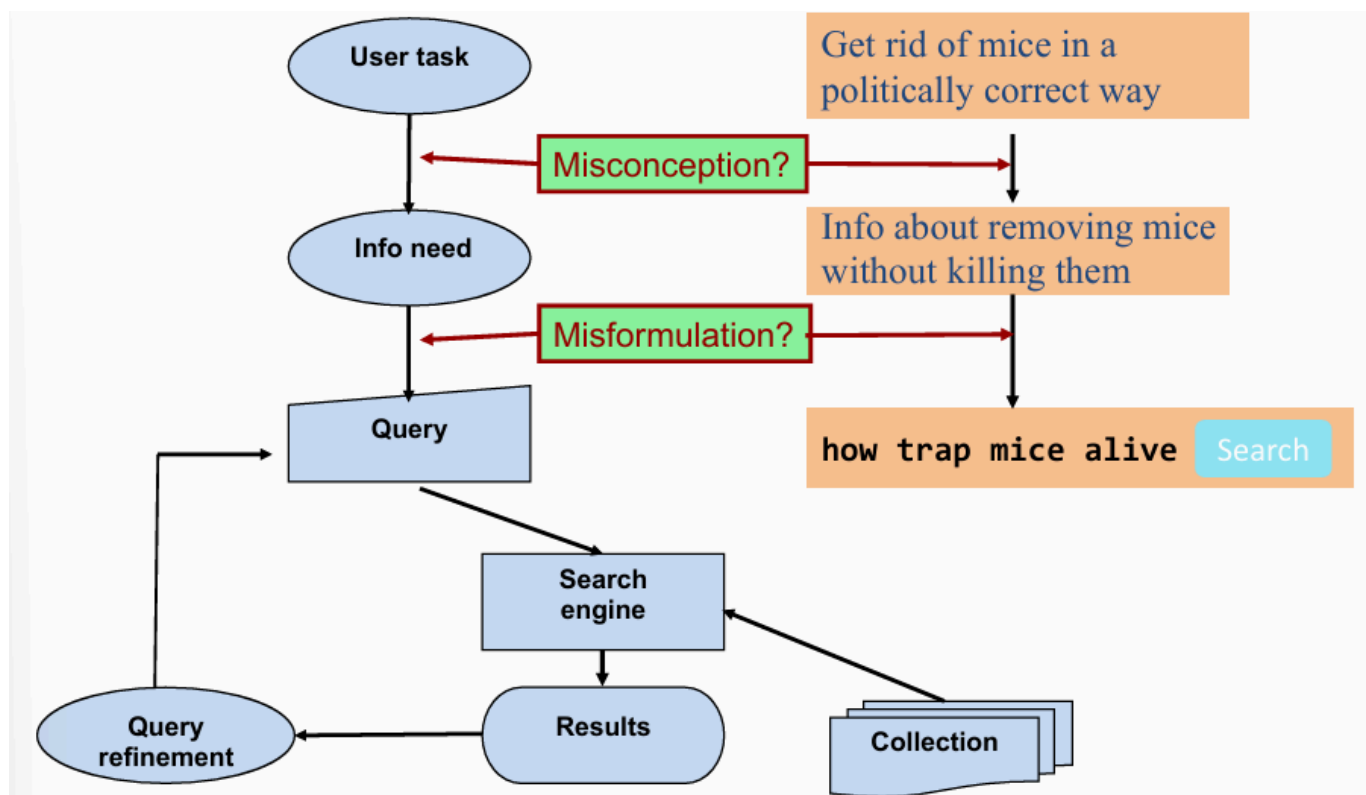
IDF rappresenta la rarità di una parola nell'intera collezione di documenti, più è rara, più è importante per identificare un documento specifico.

Riassunto (Concetti Fondamentali)

L'**information Retrieval** si basa sul trovare **materiale** (solitamente documenti) di natura **non strutturata** (solitamente testo) **a partire da una grande collection**, che soddisfino il **bisogno informativo** dell'utente.

- **Collection**: Insieme di documenti (per ora assumiamo che siano statici).
- **Goal**: Trovare i documenti che abbiano informazioni **rilevanti** per il bisogno informativo dell'utente, e che quindi lo aiutino a completare un **task**.

Il classico modello adottato in un motore di ricerca è il seguente:



Al fine di risolvere un task, lo user ha un information need e per soddisfarlo scrive una **query** nel motore di ricerca. Si noti che tra "user task" ed "info need" può esservi una **misconception** (ossia, lo user potrebbe non essere in grado di esprimere correttamente il proprio bisogno informativo) e quindi una **misformulation** tra info need e query.

Una volta fatta la query, il **search engine** estrae (qui l'IR) i documenti più rilevanti e quindi mostra i risultati all'utente, che se non soddisfatto può fare **query refinement**.

Un concetto fondamentale che approfondiremo è quello di **misurare le performance** per capire se un sistema di IR è buono o meno. Le due metriche più importanti in questo senso sono:

- **Precision:** percentuale di documenti recuperati che sono rilevanti (ossia, tra i documenti che il sistema ha recuperato (TP + FP), quanti sono effettivamente rilevanti (TP)).
- **Recall:** percentuale di documenti rilevanti che sono stati recuperati (ossia, tra i documenti che sono effettivamente rilevanti (TP + FN), quanti sono stati recuperati (TP)).

Binary Term-Document Incidence Matrix

Supponiamo di avere una grande collezione con tutte le opere di Shakespeare. Supponiamo di voler identificare quali opere contengono le parole:

- Brutus **AND** Caesar
- **NOT** Calpurnia

Un primo approccio è quello di leggere tutti i documenti, **selezionando** quelli che contengono le parole *Brutus* e *Caesar* e **scartando** quelli che contengono *Calpurnia*.

Questa alla fine è una sorta di **scansione lineare** tra i documenti, ritornando i soli documenti che rispettano l'**espressione logica** desiderata. Questo processo è noto come **grep** dei documenti.

Finché siamo in un solo computer di un utente, il comando **grep** va più che bene. Invece, su sistemi con un'enorme quantità di dati (come il web), si richiede di più:

1. Processare un'enorme quantità **molto** velocemente (frazione di secondi). E purtroppo sul web un approccio lineare come il *grepping* indurrebbe risultati disastrosi.
2. La possibilità di avere operazioni più sofisticate. Per esempio abbiamo l'espressione "*Romans near countryside*" dove l'operatore **NEAR** può significare "entro 5 parole" oppure "nella stessa frase". Questo operatore è impraticabile con il *grepping*.
3. Ottenere un **ranking** del risultato della query, in base alla "*significatività*" dei documenti rispetto a quanto richiesto.

Un primo modo per evitare uno **scanning lineare** dei documenti è quello di **indicizzare** a priori ogni documento. Per ogni documento a disposizione identifichiamo l'insieme di tutte le parole, o **termini**. Costruiamo poi una matrice $M \times N$ dove ad ogni riga è associato un termine, e ad ogni colonna un documento. In tale matrice, poniamo la cella (i, j) pari a 1 se il termine i -esimo è contenuto nel documento j -esimo, altrimenti 0. Questa è detta **matrice di incidenza**.

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Brutus AND Caesar BUT NOT Calpurnia

1 if play contains word, 0 otherwise

Perciò, per ottenere i documenti che rispettano l'espressione

Brutus AND Caesar BUT NOT Calpurnia

basta prendere la riga **indicizzata** dai termini *Brutus* e *Caesar*, poi prendere la riga indicizzata da *Calpurnia* ma **complementata**, per poi fare il **Bitwise AND**.

$$\begin{array}{r}
 110100 \quad \text{AND} \\
 110111 \quad \text{AND} \\
 101111 \\
 \hline
 = 100100
 \end{array}$$

Alla fine i documenti interessati saranno quelli per i quali il relativo bit è pari ad 1, nel nostro caso i documenti *Antony and Cleopatra* e *Hamlet*.

Questo approccio è detto **Boolean retrieval model**. In tale modello le query sono della forma di **espressioni booleane di termin**, dove i termini sono combinati tramite gli **operatori logici AND , OR e NOT**.

Esempio:

$N = 1$ milione di documenti, dove ogni documento ha circa 1000 parole. Immaginiamo che in media ogni parola richieda 6 byte $\Rightarrow 6 \cdot 1000 \cdot 1 \text{ milione} = 6 \text{ GB}$ di dati nei documenti.

Ipotizzando ora un totale di 500k parole distinte, la matrice di incidenza avrebbe 500k righe e 1 milione di colonne, con un totale di 500 miliardi di celle. Tuttavia questa matrice non avrebbe più un miliardo di 1 (considerando che ogni documento ha circa 1000 parole), il che significa che sarebbe estremamente sparsa (99,8% di celle con valore 0).

Tuttavia, c'è un problema: se la collezione di documenti è molto grande, la matrice di incidenza può diventare estremamente grande e sparsa (la maggior parte delle celle saranno 0). Questo rende l'approccio inefficiente sia in termini di spazio che di tempo.

Efficienza - Tempo

L'applicazione di un **operatore** booleano è **lineare** nel numero N di documenti. La risoluzione di una espressione ha quindi una complessità di $O(kN)$, dove k è il numero di operatori nella query.

Per quanto riguarda invece la ricerca delle righe necessarie per la query è lineare nel numero di termini M . Perciò abbiamo una complessità di $O(kN + kM)$.

Tale complessità non è per niente ragionevole, in quanto M può crescere a dismisura.

Perciò si può pensare di **ordinare** i termini in fase di costruzione della matrice d'incidenza. In tal caso si può effettuare una **ricerca dicotomica** dei termini, ottenendo una complessità di $O(kN + k \log M)$ (decisamente più ragionevole).

Efficienza - Spazio

Purtroppo in termini di **spazio** tale struttura dati non è praticabile. Infatti per larghe collezioni di documenti la matrice quasi sicuramente potrebbe non entrare in RAM, mentre per piccole collezioni conviene fare il *grepping*. Infatti, supponiamo di avere $N = 500k$ documenti ed $M = 1M$ di termini. Se per ogni cella abbiamo bisogno di un bit, avremo una matrice grande $NM = 500,000,000,000$ di bit, ovvero $62,500,000,000 \text{ GB} \approx 58 \text{ GB}$.

Inverted Index

Un'osservazione che si può fare è che la matrice d'incidenza per molti **termini d'interesse** (nomi, luoghi, ecc...) è piena di zeri, ovvero **sparsa**.

Certamente per molti termini, come gli articoli, ci sono molti 1. Però generalmente non si fanno query in cui richiedo tutti i documenti con il termine "the", perché certamente tutti i documenti saranno compresi nella risposta.

Perciò per ottimizzare possiamo pensare di usare un **dizionario** di termini (detto anche **vocabolario** o **lexicon**), dove per ogni termine conserviamo una **lista di documenti** nei quali occorrono. Tale lista è anche detta **posting list**.

Nella costruzione di un indice inverso, assumiamo che viene costruito in modo tale che sia i termini sia le loro posting lists sono mantenute **ordinate**.

Anche se in termini di efficienza è dispendioso mantenere l'ordinamento a fronte di **inserimenti** e **rimozioni**.

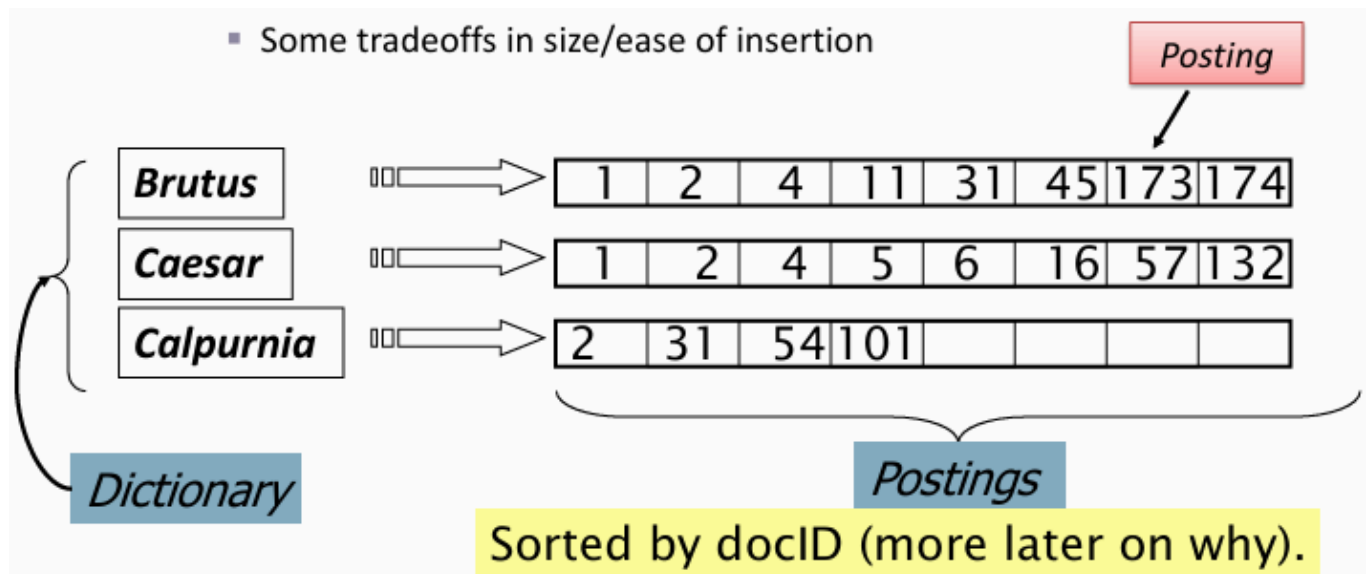
Però nell'ambito dell'IR non si vuole gestire un numero elevato di inserimenti, rimozioni o update.

Quindi, un **Inverted Index** consiste nel: per ogni termine t , memorizziamo una lista di tutti i documenti che lo contiene. Ogni documento è identificato da un docID unico, e la lista degli docID è chiamata **posting list** di quel dato termine (e ogni documento è una posting).

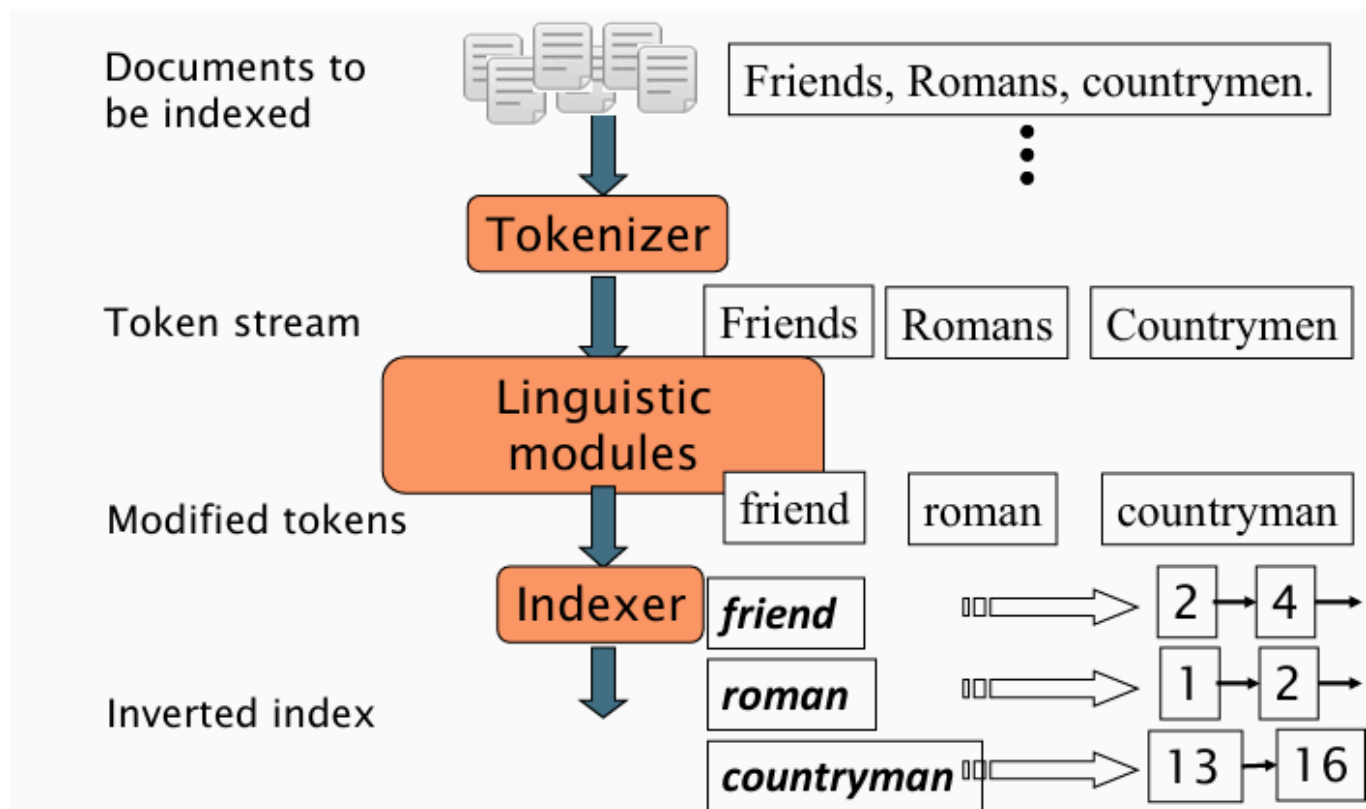
Dal momento che una parola potrebbe apparire in molti documenti mentre un'altra in pochissimi, è necessario avere delle posting list di lunghezza variabile:

- su disco, le posting list sono memorizzate in modo contiguo (per permettere letture sequenziali efficienti), esattamente come si vede nell'immagine (dal momento che in disco restano statiche).
- in memoria RAM invece si possono usare strutture dati più flessibili (es. linked list, array dinamici) per gestire le posting list di lunghezza variabile.

Per gestire il tutto, si sfrutta un **Dizionario** con chiavi i termini e valori le posting list. Se volessi cercare la posting list di una parola, dovrei anzitutto cercare la parola nel dizionario ($O(\log n)$ se i termini sono ordinati) per poi accedere alla posting list associata ($O(1)$)



Inverted Index: Costruzione



Si parte da una collezione di documenti e si usa una serie di tecniche, che però devono essere utilizzate con cautela e in base al contesto.

1. **Tokenization:** Data la sequenza di caratteri che compongono un documento, il task della **tokenizzazione** è il task che consiste nello "spezzare" tale sequenza in pezzi (detti **token**). Generalmente questi token sono le parole che compongono le frasi, perciò generalmente si utilizzano gli spazi e la punteggiatura come separatori.

- **Input:** *Friends, Romans, Countrymen, len me your ears;*
- **Output:** *Friends, Romans, Countrymen, lend, me, your, ears.*

Un **token** è un'istanza di una sequenza di caratteri in un particolare documento che sono raggruppati come un'utile unità semantica per l'elaborazione. Per esempio "*Micheal Jackson*" è meglio trattarlo come un **singolo token** *Micheal Jackson* perché è più utile al task di IR.

Un **tipo** è la classe di tutti i token che contengono la stessa sequenza di caratteri. Nel nostro caso possiamo avere il tipo:

- *Micheal Jackson , Micheal jackson, micheal Jackson, micheal jackson, ...*

Un **termine** è invece un elemento di un tipo, generalmente **normalizzato**, il quale viene usato nel nostro motore di IR.

2. **Linguistic Modules:** dopo la tokenizzazione, si possono applicare diverse tecniche linguistiche per migliorare la qualità dell'indice. Ad esempio:

- **Normalization:** La normalizzazione dei token è il processo di canonizzazione dei token, ovvero si riducono i token e si cerca di raggruppare token differenti che rappresentano una stessa cosa in un unico token. Per esempio, io voglio che se ricerco *U.S.A* oppure ricerco *USA* ottengo lo stesso risultato dal mio motore di IR. Il modo più comune di standardizzare è quello di definire **classi di equivalenza** tra termini. Se per esempio entrambi i termini "anti-discriminatory" e "antidiscriminatory" sono entrambi mappati nel termine "antidiscriminatory", allora tutte le query che li comprendono avranno come risultato lo stesso insieme di documenti.
- **Stop Words:** Ci sono termini che sono contenuti in ogni documento, come per esempio gli articoli, le congiunzioni, le proposizioni, ecc... Per esempio, se ricerco la frase "President of the United States" gli unici termini realmente utili alla ricerca sono "President" e "United States". Infatti, molto probabilmente la mia ricerca sarà un query del tipo $\text{president} \wedge \text{united states}$. Perciò, possiamo pensare di rimuovere tutte le parole con altissima frequenza in una lingua, e che non hanno utilità ai fini di una ricerca.
- **Stemming:** riduzione delle parole alla loro radice (es: authorize, authorization, authorized diventano "author"). **Nota Bene:** ci possono essere problemi ad esempio "Un giorno feci una partita", feci si confonde tra fare e feci.

3. Dopo queste trasformazioni otteniamo i **Modified Tokens**. Solo a questo punto entra in gioco l'**indexer**, che costruisce l'**inverted index** a partire dai modified tokens.

Una volta ottenuti i modified tokens, l'indexer costruisce l'inverted index associando ogni token ai documenti in cui appare. Anzitutto si creano le coppie (term, docID) per ogni token e documento in cui appare. Successivamente, si ordinano queste coppie per termine e docID, in modo da poter costruire efficientemente le posting list. Dopo il sort si ottiene quindi qualcosa del tipo:

```
(Brutus, 1)
(Brutus, 2)
(Caesar, 1)
(Caesar, 2)
(Caesar, 2)
(enact, 1)
(Julius, 1)
```

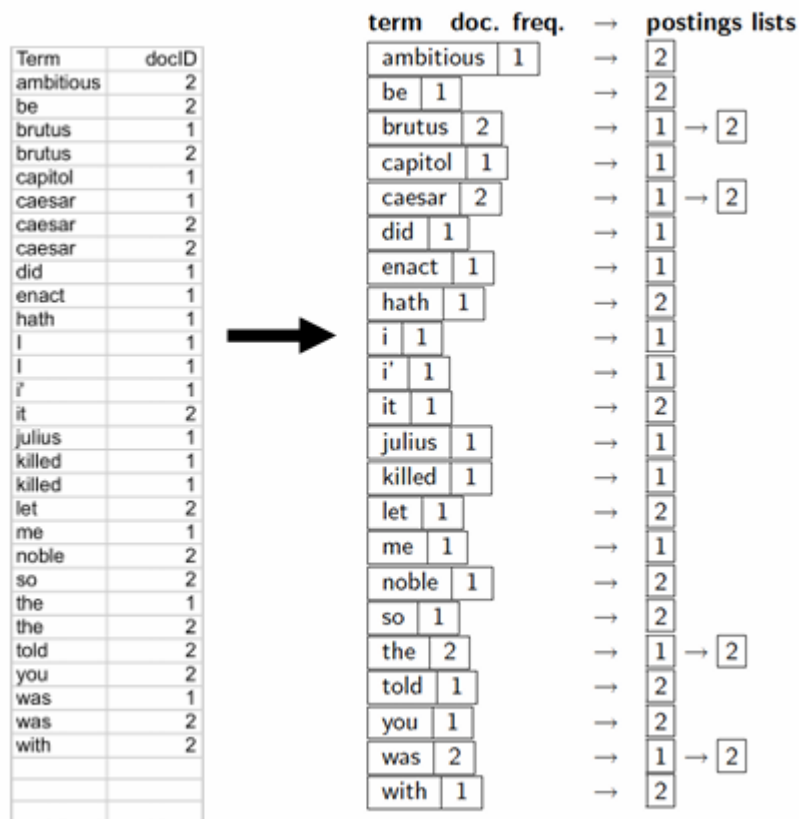
Dopodiché i dati sono separati in due strutture:

- **Dizionario**, che contiene tutti i termini distinti (es: Brutus, Caesar, enact, Julius, ecc...). **Inoltre viene anche salvata la document frequency, ossia per ogni termine il numero di documenti in cui appare.** La frequenza sarà molto importante per fare ranking dei risultati (termini più rari sono più importanti per identificare un documento specifico),

ottimizzare le query (es: se una query contiene un termine molto raro, è più efficiente partire da quello) e calcolare TF - IDF.

- **Posting list**, che contiene per ogni termine la lista dei documenti in cui appare (es: per Brutus → [1, 2] , per Caesar → [1, 2, 2], ecc...).

Si noti che se una parola appare più volte in un documento, questa viene registrata una sola volta nella posting list.



In questo senso, in memoria si spende principalmente per mantenere:

- I termini distinti nel dizionario,
- Le posting list per ogni termine, che contengono i docID dei documenti in cui il termine appare,
- I puntatori alle posting list, che permettono di accedere efficientemente alle posting list a partire dal dizionario.

Inverted Index : query processing

Ora ci si chiede come processare una query usando l'inverted index. Supponiamo di avere una query del tipo "Brutus AND Caesar". Come già anticipato per processare una query di questo tipo dobbiamo anzitutto accedere alle posting list di Brutus e Caesar, e poi fare un **merge** tra le due posting list per trovare i documenti che contengono entrambi i termini (si scorrono le due posting list in parallelo e si avanza in quella che ha il docID più piccolo fino a trovare un match o superare l'altro).

Ipotizzando che la prima lista abbia x elementi e la seconda y , il tempo di esecuzione del merge è $O(x + y)$ (in quanto si scorre ogni lista al massimo una volta). **Ovviamente per far sì che il merge sia possibile è cruciale che i docID siano ordinati all'interno delle posting list.**

```
INTERSECT(p1, p2):  
  answer ← {}  
  
  while p1 ≠ NIL and p2 ≠ NIL  
    if docID(p1) = docID(p2)  
      add docID to answer  
      p1 ← next(p1)  
      p2 ← next(p2)  
  
    else if docID(p1) < docID(p2)  
      p1 ← next(p1)  
  
    else  
      p2 ← next(p2)  
  
  return answer
```

Boolean Retrieval Model

Si passa quindi ad uno dei modelli più semplici di IR, ossia il **Boolean Retrieval Model**. In questo modello, una query è espressa come una formula booleana composta da termini e operatori logici (AND, OR, NOT). Il sistema restituisce i documenti che soddisfano la formula booleana. Ad esempio "Brutus OR NOT Calpurnia" restituirebbe tutti i documenti che contengono Brutus o che non contengono Calpurnia.

Questo modello ha due caratteristiche principali:

- Considera i documenti semplicemente come insieme di parole, senza considerare l'ordine e la grammatica (Bag of Words).
- Il sistema non considera quanto spesso una parola compare, né la sua posizione all'interno del documento. L'unica cosa che conta è se la parola è presente o meno all'interno del documento. Si parla in questo senso di **Exact Match**: un documento viene restituito se e solo se soddisfa esattamente la formula booleana della query.

Questo approccio è stato per anni il modello dominante nei sistemi di IR, ma ha dei limiti evidenti. Infatti è molto poco flessibile e non tiene conto della rilevanza dei documenti rispetto alla query, non permettendo quindi di fare ranking (tutti i documenti che soddisfano la formula

booleana vengono restituiti, indipendentemente da quanto siano rilevanti). **Boolean è quindi assolutamente inadatto per motori di ricerca generici come Google.**

Tuttavia ancora oggi molti sistemi di IR usano ancora Boolean, come ad esempio macOS Spotlight, Email Search, Library catalogues. Infatti l'exact match permette di avere risultati molto precisi ed è **utile quando l'utente sa esattamente cosa sta cercando.**

Nota Bene: Ricordando che la Precision è il numero di documenti rilevanti su tutti i documenti restituiti mentre la Recall è il numero di documenti rilevanti restituiti su tutti i documenti rilevanti, il Boolean Retrieval Model tende ad avere una Precision molto alta (in quanto restituisce solo documenti che soddisfano esattamente la formula booleana, quindi mi aspetto che molti dei documenti restituiti siano rilevanti) ma una Recall più bassa (in quanto alcuni documenti rilevanti potrebbero non soddisfare esattamente la formula booleana e quindi non essere restituiti).

Un famoso esempio di applicazione del Boolean Retrieval Model è **WestLaw**: si tratta di un database giuridico che contiene milioni di documenti legali (sentenze, leggi, ecc...) e che permette agli avvocati di cercare documenti rilevanti per i loro casi. WestLaw utilizza un sistema di IR basato su Boolean Retrieval Model, in quanto gli avvocati spesso sanno esattamente cosa stanno cercando (es. una sentenza specifica) e quindi l'exact match è particolarmente utile in questo contesto.

Implementi Boolean Queries and Optimization

Vediamo ora come adattare il merge per gli operatori logici (OR, NOT). Per l'OR, l'idea è di restituire tutti i documenti che appaiono in almeno una delle due posting list. Quindi si scorre in parallelo le due posting list e si aggiunge al risultato ogni documento che appare in almeno una delle due liste. Il tempo di esecuzione è sempre $O(x + y)$.

```
UNION(p1, p2):  
  answer <- {}  
  
  while p1 ≠ NIL and p2 ≠ NIL  
  
    if docID(p1) = docID(p2)  
      add docID to answer  
      p1 <- next(p1)  
      p2 <- next(p2)  
  
    else if docID(p1) < docID(p2)  
      add docID(p1) to answer  
      p1 <- next(p1)  
  
  else
```

```
    add docID(p2) to answer
    p2 <- next(p2)

return answer
```

Per il NOT dobbiamo considerare tre casistiche:

- **NOT A:** restituisce tutti i documenti che non contengono A . In questo caso si scorre la posting list di A e si aggiungono al risultato tutti i documenti che non appaiono in quella lista. Bisogna quindi conoscere tutta la collezione di documenti per poter restituire quelli che non contengono $A \implies O(N)$, dove N è il numero totale di documenti nella collezione.
- **A AND NOT B:** restituisce tutti i documenti che contengono A ma non B . In questo caso si scorre la posting list di A e si aggiungono al risultato solo i documenti che non appaiono nella posting list di $B \implies$ è sufficiente riadattare il merge dove:
 - scorriamo entrambe le liste
 - se troviamo un elemento in comune lo scartiamo
 - quando un docID è solo in A lo aggiungiamo al risultato. Il tempo di esecuzione è $O(x + y)$, dove x è la lunghezza della posting list di A e y è la lunghezza della posting list di B .
- **A OR NOT B:** restituisce tutti i documenti che contengono A o non contengono B . Qui ci servono tutti i documenti che non contengono B , quindi è necessario conoscere tutta la collezione di documenti. Una volta ottenuti i documenti che non contengono B , dobbiamo fare l'OR con la posting list di A , per un tempo totale di $O(N)$.

Man mano che le query si fanno più complesse, è necessario fare più merge tra le posting list diverse (es: Brutus OR Caesar) AND NOT (Antony OR Calpurnia)). In questi casi è possibile **ottimizzare l'esecuzione della query** scegliendo l'ordine in cui fare i merge.

Vediamo un esempio:

Brutus AND Calpurnia AND Caesar, dove Brutus : 1 2 3 5 8 16 21 34, Caesar : 2 4 8 16 32 64 128, Calpurnia : 13 16.

Si osserva che Calpurnia è il termine più raro (appare in meno documenti), e dato che quando si effettua il merge tra A e B il risultato non può essere più grande della posting list più corta, è più efficiente partire da Calpurnia e fare prima il merge con Brutus, per poi fare il merge con Caesar. In questo modo si riduce il numero di documenti da considerare nei merge successivi, ottimizzando l'esecuzione della query.

Quindi nel nostro caso Calpurnia AND Brutus AND Caesar si esegue come (Calpurnia AND Brutus) AND Caesar, e in generale **per eseguire query con molti AND bisogna intersecare prima le posting lists dei termini più rari**. Come capisco che un termine è più raro di un

altro? Semplice, **guardando la document frequency (DF) che è salvata nel dizionario per ogni termine** (per questo prima l'abbiamo salvata).

E se avessi una query del tipo (A OR B) AND (C OR D) AND (E OR F)? In questo caso si procede ugualmente a prima, solo che si deve stimare il numero di documenti per ogni OR: usando un approccio conservativo, si può stimare che il numero di documenti restituiti da un OR sia al massimo la somma delle lunghezze delle posting list dei termini coinvolti (es. $\text{dim}(A \text{ OR } B) = \text{df}(A) + \text{df}(B)$).

E se invece avessi una query del tipo **friends AND romans AND (NOT countrymen)**? Come possiamo usare la frequenza di countrymen? In questo caso bisogna ragionare "al contrario": se countrymen è un termine molto raro, allora romans AND NOT countrymen restituirà molti documenti, quasi pari alla dimensione della posting list di romans. Al contrario, se countrymen è un termine molto comune, allora romans AND NOT countrymen restituirà pochi documenti, in quanto la maggior parte dei documenti che contengono romans conterranno anche countrymen. Quindi se countrymen è termine raro conviene eseguire prima friends AND romans per ridurre il numero di candidati, mentre se countrymen è termine comune conviene eseguire prima romans AND NOT countrymen. (Ricorda che l'ottimizzazione delle query boolean è basata sulla regola di fare prima le operazioni che restituiscono meno documenti, in modo da ridurre il numero di documenti da considerare nei merge successivi).

Phrase Queries and Positional Indexes

Molti termini tecnici o complessi sono organizzati in sequenza di parole o intere frasi.

Quello che vogliamo è trattare la query "Stanford University" in modo tale che frasi del tipo "Then inventor Stanford Ovshinsky never went to university." non vengano considerate come risultato.

Queste query sono anche dette **phrase query**.

Un primo approccio per il nostro problema è quello di considerare ogni coppia di termini **consecutivi** in un documento come una singola frase. Tali coppie di termini sono anche detti **biword**.

Per esempio, se abbiamo il documento "Friends, Romans, Countrymen" generiamo le biword:

- friends romans
- romans countrymen

In tale modello la query "stanford university palo alto" verrà trattata come:

$\text{stanford university} \wedge \text{university palo} \wedge \text{palo alto}$

Ci aspettiamo che questa query funzioni bene in pratica, ma può incorrere in svariati **falsi positivi**. Infatti senza esaminare il contenuto dei documenti, non si può verificare che il risultato della query ritorni tutti i documenti che contengano l'intera sequenza **consecutiva** delle 4 parole.

Per evitare falsi positivi dovrei indicizzare n-grammi più grandi, ma ciò porterebbe a un indice molto grande e poco efficiente.

La soluzione al problema, usata nei motori di ricerca moderni, è il **positional index**. L'idea è di mantenere per ogni termine non solo la lista dei documenti in cui appare, ma anche le posizioni all'interno di ogni documento in cui il termine appare. In questo modo, quando voglio cercare una frase esatta, posso verificare se i termini della frase appaiono nelle posizioni corrette all'interno dei documenti.

Vediamo un esempio per capire come si verifica una, "to be or not to be", usando un positional index. L'idea è semplicemente di prendere le posting list di tutti i termini della frase (to, be, or, not) e poi verificare se nei documenti in cui appaiono i termini, questi appaiono in posizioni consecutive.

- **to:**

- 2:1,17,74,222,551; 4:8,16,190,429,433; 7:13,23,191; ...

- **be:**

- 1:17,19; 4:17,191,291,430,434; 5:14,19,101; ...

Nell'immagine sopra si vede come il doc4 possiede un "to be" dal momento che in posizione 15 ha to e 17 be. Come verificare nel pratico? **Si utilizza di nuovo il merge**: anzitutto ovviamente si usa il merge per trovare i documenti che contengono tutti i termini della frase (to, be, or, not). Successivamente si fa il **merge sulle posizioni**: la procedura è identica solo che si aggiunge il documento tra i candidati solo se i termini appaiono in posizioni consecutive.

I positional index inoltre permettono di gestire query più complesse oltre alle phrase queries booleane, come le **proximity queries**. Queste sono queries che chiedono che una certa parola sia in prossimità k rispetto ad un'altra parola. Ad esempio STATUTE /3 FEDERAL significa che la parola STATUTE deve apparire entro 3 parole da FEDERAL. Per gestire questo tipo di query, si fa un merge simile a quello delle phrase queries, ma invece di cercare posizioni consecutive ($pos2 = pos1 + 1$), si cerca che la differenza tra le posizioni sia minore o uguale a k ($|pos2 - pos1| \leq k$).

Es. statute: 10, 40, 80 federal : 12, 70, statute /3 federal, confrontiamo prima 10 vs 12: $|12 - 10| \leq 3$? sì, quindi abbiamo trovato un match. Poi scorriamo statute (perché $10 < 12$) e

confrontiamo 40 vs 12: $|12 - 40| \leq 3$? no, quindi scorriamo federal etc...

Si noti che modificando il merge come descritto, il costo resta lineare nelle due liste di posizioni.

Ora ci chiediamo, **ma quanto ci costa aggiungere tutti questi indici di posizioni?**

Parecchio. Dovendo salvare per ogni parola tutte le posizioni, il numero totale di posizioni salvate è pari al numero totale di parole in tutti i documenti, che è molto più grande del numero di termini distinti. Tuttavia salvare una parola richiede più byte rispetto a salvare una posizione, dal momento che una posizione può essere rappresentata con un numero intero mentre una parola richiede in media 6 byte.

La dimensione del positional index dipende molto dalla dimensione dei documenti: immaginiamo che una parola rara appare con frequenza 0.1% (una volta ogni 1000 parole). Allora, in un documento di 1000 parole, quella parola apparirà in media una volta, quindi dovremmo salvare una posizione per quel documento. In un documento di 100.000 parole, quella parola apparirà in media 100 volte, quindi dovremmo salvare 100 posizioni per quel documento. Quindi più i documenti sono lunghi, più il positional index diventa grande.

Rules of thumb:

- In generale, un positional index è circa 2-4 volte più grande di un inverted index classico (term --> list of docs).
- La dimensione dell'indice posizionale è circa 35-50% della dimensione dei documenti originali (in termini di byte), quindi è un indice molto grande, ma che permette di gestire query molto più complesse e precise rispetto all'inverted index classico. Es: se il corpus originale è di 100 GB, solo l'indice posizionale potrebbe essere di 35-50 GB.

Nonostante il costo significativo, oggi quasi tutti i motori di ricerca usano indici posizionali, in quanto permettono di gestire query molto più complesse e precise, migliorando significativamente la qualità dei risultati restituiti agli utenti.

Schema ibrido: alcune frasi sono molto comuni, per esempio "Micheal Jackson", "New York", "The Who". Se usassimo il positional index per cercarle, dovremmo ogni volta fare il merge delle posizioni, e questo può essere molto lento. Per questo motivo alcuni sistemi fanno un indice misto: si usa il positional index normale e in più si mantengono quelle frasi comuni come termini distinti nell'inverted index classico. In questo modo quando si cerca la frase in questione può essere restituita direttamente la posting list ad essa associata senza fare operazioni di merge sulle posizioni.

Uno studio mostra come usando questo approccio, nonostante uno spazio aggiuntivo del 26% per mantenere le frasi comuni, il tempo di risposta alle query è in media 1/4 rispetto a un

sistema che usa solo il positional index. Si parla quindi di tradeoff tra spazio e tempo di risposta.